Why Does Grokking Happen?

A Mechanistic Account via Singular Learning Theory,
Computational Mechanics, and Training Dynamics

Benjamin Shaffrey
MSc Artificial Intelligence & MSc Mathematics

May 13, 2026

Abstract

We propose a research programme to give a precise, mechanistically grounded answer to the question "why does grokking happen?" Our approach synthesises four complementary frameworks: (1) the local learning coefficient (LLC) from singular learning theory, which tracks the effective complexity of the model's position in parameter space throughout training; (2) belief state geometry from computational mechanics, which tracks the formation of predictive representations in the model's activations; (3) data attribution via unrolling, which identifies which training examples drive parameter changes at each moment; and (4) dynamical systems analysis of the training trajectory, which characterises the basin structure and identifies the bifurcation governing the transition. The dynamical-systems analysis identifies the lazy and rich regimes as competing attractors of the training dynamics and grokking as the bifurcation between them, while the LLC, belief geometry, and unrolling each provide a complementary diagnostic of the attractors and the trajectory between them. We study grokking primarily in a minimal transformer trained on modular addition, where the learned algorithm is mechanistically understood and the computational mechanics framework has been recently developed, and secondarily in a two-layer MLP on the same task, where belief geometry serves as the primary tool for algorithm identification.

Our central thesis is that grokking is a transition from the lazy regime (where the Neural Tangent Kernel is approximately fixed and the model memorises via kernel interpolation) to the rich regime (where features are learned and the model discovers task-adapted representations). This transition can be triggered by any mechanism that induces feature learning; weight decay, small output scale, balanced initialisation. This motivates the hypothesis that several distinct hyperparameters act through a shared physical channel (NTK evolution) and could in principle be reduced to a small number of *effective* control parameters governing the bifurcation. Whether this dimension reduction holds, and at what effective dimension, is one of the empirical questions the project aims to substantiate or refute via the bifurcation analysis in section 4.7. The LLC should track this transition universally: high in the lazy regime, dropping during the transition, low at the generalising solution. Beyond explaining grokking, the project aims to establish belief state geometry as a model-agnostic tool for identifying learned algorithms, and to investigate whether SGD in singular models has an intrinsic simplicity bias expressible in terms of the LLC.

Contents

1 Introduction and Motivation **3**

2 Background **4**

2.1 The Model and Task 4
2.2 The Lazy-to-Rich Transition 5
2.3 The Learned Algorithm: From Fourier Modes to Group Representations 6
2.4 Computational Mechanics Perspective 7

2.5 Singular Learning Theory and the Local Learning Coefficient 8
2.6 Data Attribution via Unrolling 8
2.7 Dynamical Systems Framework 9

3 Research Questions **10**

4 Methodology **11**
4.1 Experimental Setup 11
4.2 Step 1: Belief Geometry as Algorithm Identification 11
4.3 Step 2: LLC Tracking During Training 14
4.4 Step 2b: Compression-Based Complexity Triangulation 15
4.5 Step 3: Data Attribution via Unrolling 15
4.6 Step 4: Synthesis via Unrolling-Informed Data-Refined LLC 15
4.7 Step 5: Dynamical Systems Analysis and Bifurcation 16

5 Expected Outcomes **18**

6 Broader Research Programme **19**

1 Introduction and Motivation

Grokking (the phenomenon whereby a neural network first memorises its training data and only much later generalises to unseen examples) was first identified by Power et al. [2022] in small transformers trained on algorithmic tasks including modular addition. Despite substantial subsequent work characterising the *what* of grokking (which circuits form, what progress measures track the transition), a precise answer to *why* grokking occurs (why the transition is delayed, why it is sharp, and what mechanistically drives it) remains open.

Recent work by Kumar et al. [2024] has significantly reframed the problem. They demonstrate that grokking can occur *without* weight decay and with increasing parameter norm, both in a two-layer MLP on polynomial regression and on modular addition with a one-layer transformer. The two fundamental control parameters they identify are (1) the output scale α, which controls the rate of feature learning (large α pushes the model into the lazy/kernel regime, delaying grokking), and (2) NTK-task alignment, which controls how much feature learning is necessary. Weight decay is one of several mechanisms that can trigger the lazy-to-rich transition, it forces the NTK to evolve [Lewkowycz and Gur-Ari, 2020], but it is neither special nor necessary. Dominé et al. [2025] provide exact analytical solutions for the learning dynamics in deep linear networks as a function of the relative scale parameter λ (the difference in weight covariance between layers), showing that $\lambda = 0$ yields rich/sigmoidal dynamics while $|\lambda| \to \infty$ yields lazy/exponential dynamics. Their work demonstrates that the transition between regimes is controlled by a complex interaction of architecture, relative scale, and absolute scale.

We propose to study grokking in two settings:

(i) **A single-layer, single-head transformer trained on modular addition modulo a prime p.** This setting is uniquely favourable because the learned algorithm is mechanistically understood [Nanda et al., 2023, Chughtai et al., 2023, Yip et al., 2024, Wu et al., 2024]: the model implements a Fourier-based algorithm. Poncini [2025] has provided a computational mechanics characterisation of this algorithm, identifying the precise predictive vectors linearly represented at each position in the residual stream. The model is small ($\sim$226k parameters) and the dataset is finite (p^2 examples), making rigorous analysis feasible.

(ii) **A two-layer MLP on the same modular addition task.** Two MLP variants are relevant to this programme. (i) A Chughtai-style two-input concatenation MLP [Chughtai et al., 2023], with per-token embedding matrices W_a, W_b, pre-activation $h = W_a[a] + W_b[b]$, nonlinearity σ, and a linear unembedding to p logits. This is the natural architecture for belief-geometry comparison with the transformer because it shares the "two-input concat $\to$ hidden $\to$ unembed" structure of the transformer's MLP block. (ii) A Kumar-style single-input MLP $f(x;\theta) = \alpha \cdot W_2\sigma(W_1x)$ [Kumar et al., 2024], which exposes an output-scale parameter α controlling the lazy-to-rich transition and is the natural architecture for the bifurcation analysis of section 4.7. In both, the analytical RLCT is more tractable than for the transformer (closer to existing results on layered neural networks by Watanabe and Aoyagi). Crucially in either variant, the learned algorithm *cannot* be read from the weights as cleanly as in the transformer, making belief geometry the primary tool for algorithm identification, and thus a strong test of its model-agnostic applicability.

This project can be understood as a concrete instantiation of the "S4 correspondence" articulated by Pepin Lehalleur et al. [2025a]: *statistical structure* in the data (the group-theoretic structure of modular addition) shapes the *geometric structure* of the loss landscape (singularity structure measured by the LLC), which determines the *developmental structure* of the learning process (the lazy-to-rich transition), which produces the *algorithmic structure* in the learned model (the Fourier algorithm identified by belief geometry). Our contribution is to track all four levels simultaneously in a single, controlled setting.

Our central hypothesis is:

Hypothesis 1 (Lazy-to-Rich Transition Hypothesis)**.** *Grokking occurs because the model begins training in the lazy regime; where the NTK is approximately fixed, the model memorises via kernel interpolation, and the LLC is high.Any mechanism that forces feature learning (weight decay, small output scale α, balanced initialisation, appropriate learning rate) destabilises this lazy regime, triggering a transition to the rich regime where the model discovers task-adapted representations with lower LLC. The memorisation phase corresponds to the lazy regime; the delay is the time for feature learning to begin; the sharpness reflects self-reinforcing feature learning once it starts.*

We hypothesise that the bifurcation governing grokking is parameterised, at leading order, by a small number of effective control parameters rather than by each hyperparameter independently. The minimal version of this hypothesis posits an *effective laziness* L (some function of weight decay κ, output scale α, initialisation scale σ_{init}, and λ-balancedness) together with the NTK-task alignment A, which together would span a 2D effective control space inside the higher-dimensional hyperparameter space.

This hypothesis is suggested but not established by the existing literature. Kumar et al. [2024] show that grokking occurs without weight decay (so κ is not essential) and identify α and NTK-task alignment as the two primary controls, suggesting κ and α may act through a common NTK-evolution channel. Lewkowycz and Gur-Ari [2020] show weight decay forces NTK evolution at rate $O(\kappa)$, a specific shared mechanism. Dominé et al. [2025] provide exact dynamics in deep linear networks parameterised by λ-balancedness, plausibly acting through the same channel. None of this constitutes a demonstrated collapse of $(\kappa, \alpha, \sigma_{\text{init}}, \lambda)$ onto a single effective axis, nor evidence that we have enumerated all relevant axes (learning rate, batch size, architecture, task structure could each contribute). The hypothesis is motivated by a plausible shared mechanism rather than established by direct measurement, and one of our aims (section 4.7) is to test it.

2 Background

2.1 The Model and Task

We study three architectures, in two tiers of analytical role: a *calibration tier* (transformer + Chughtai-style MLP) where the learned algorithm has been mechanistically reverse-engineered in prior work and serves as ground truth, and a *bootstrap-test tier* (Kumar-style MLP) where the learned algorithm has not been pre-characterised and which therefore serves as a test of the methodology's ability to identify algorithms without prior mechanistic interpretation to lean on.

Transformer (calibration tier). A single-layer, single-head transformer with residual stream dimension $d_{\text{model}} = 128$, MLP hidden dimension $d_{\text{mlp}} = 512$, vocabulary size $|\mathcal{X}_p| = p + 1$ (including a BOS token), and context length $n_{\text{ctx}} = 3$. The model is trained on sequences $\texttt{BOS}\, a\, b\, c$ where $c = a + b \bmod p$, with cross-entropy loss computed only on the final token position. Following Power et al. [2022], we use $p = 113$ and train with full-batch gradient descent using AdamW (learning rate 10^{-3}, weight decay 1.0). Training proceeds for 25,000 epochs. The transformer's learned algorithm has been extensively mechanistically reverse-engineered [Nanda et al., 2023, Chughtai et al., 2023, Yip et al., 2024, Wu et al., 2024, Poncini, 2025], giving ground truth against which the belief-geometry methodology can be validated.

Chughtai-style two-input MLP (calibration tier). A Chughtai-style [Chughtai et al., 2023] two-input concatenation MLP: per-token embedding matrices W_a, W_b, pre-activation $h = W_a[a] + W_b[b]$, nonlinearity σ (ReLU), linear unembedding to p logits, with hidden dimension matched to the transformer's MLP block. This architecture is the most direct counterpart to

the transformer's MLP block: cross-architecture comparisons isolate "one block vs. full residual transformer" rather than confounding it with "one-input vs. two-input". Chughtai et al. [2023] provides ground-truth mechanistic reverse-engineering of the learned algorithm (GCR specialised to the cyclic group C_p), enabling cross-architecture validation of the belief-geometry methodology against an independent characterisation of the same algorithm family. Training uses the same full-batch AdamW recipe as the transformer.

Kumar-style single-input MLP (bootstrap-test tier). A Kumar-style [Kumar et al., 2024] single-input MLP $f(x;\theta) = \alpha \cdot W_2\sigma(W_1x)$. The role of this architecture is distinct from the first two: Kumar et al. [2024] use it primarily on polynomial regression to study the lazy-to-rich transition, but the learned algorithm has not been mechanistically reverse-engineered for either polynomial regression or modular addition. The Kumar MLP is therefore used as a test of the bootstrap programme of section 6: can the methodology calibrated on the transformer and the Chughtai MLP (where ground truth is available) identify the learned algorithm in a setting where it is not? We pursue this in two stages of increasing ambition:

(i) *Kumar MLP on modular addition.* Train to grokking on the same modular-addition task as the other two architectures and apply the full methodology. The cyclic-group structure of the task constrains the algorithmic family that should plausibly be learned, so this stage functions partly as a sanity check (the methodology should identify Fourier-style multiplication) and partly as a genuine extension; the Kumar architecture is sufficiently different from both calibration-tier architectures that the diagnostic answers (mode count, α^2 concentration, Δ_{scaffold}) are not obvious a priori. Even if the precise algorithmic content is recovered straightforwardly, this is informative: it adds a third architectural data point on the compressive↔redundant Fourier-coverage spectrum and demonstrates that the methodology calibrated on the calibration tier is reproducible.

(ii) *Kumar MLP on polynomial regression.* If stage (i) succeeds, we apply the methodology to the Kumar MLP on polynomial regression: the setting in which Kumar et al originally studied it. This is a genuinely different task: the algorithmic family is not constrained by cyclic-group structure, and no prior mechanistic reverse-engineering is available. Stage (ii) is an attempt to come closer to executing the inverse-problem stage of the broader bootstrap programme (section 6). If stage (i) reveals fundamental obstacles, stage (ii) is left to follow-up work.

Output-scale parameter for the bifurcation analysis. The Kumar architecture has an explicit output-scale parameter α, which is convenient for the lazy-to-rich sweeps in section 4.7. To make the same control axis available uniformly across all three architectures, we introduce an explicit output-scale parameter to the transformer and the Chughtai MLP by multiplying the unembedding output by α. This is a minor architectural addition that does not affect the static-phase belief-geometry analyses (the converged-checkpoint geometry is unchanged up to a scalar rescaling of activations that the supervised ridge regression absorbs) but enables a clean α-sweep across all three architectures during the bifurcation analysis. The other bifurcation axes ($\kappa, \sigma_{\text{init}}, \lambda$-balancedness) are universal across architectures and require no modification. The simpler structure of both MLP variants additionally makes analytical RLCT calculations more tractable than for the transformer.

2.2 The Lazy-to-Rich Transition

The lazy regime [Chizat et al., 2019, Jacot et al., 2018] occurs when the model remains near its linearised form during training, with the NTK approximately fixed. The model behaves as

a kernel method and fits training data via kernel interpolation without learning task-specific features. This produces memorisation when the initial NTK is not well-aligned with the task.

The rich regime [Saxe et al., 2014, Dominé et al., 2025] is characterised by an evolving NTK, non-convex dynamics that navigate saddle points, sigmoidal learning curves, and simplicity biases such as low-rankness. The model learns task-adapted features.

Kumar et al. [2024] show that grokking is the transition between these regimes. Multiple factors control this transition: the output scale α (large $\alpha \to$ lazy), weight decay κ (forces NTK evolution at rate $O(\kappa)$ per Lewkowycz and Gur-Ari [2020]), initialisation scale, and λ-balancedness [Dominé et al., 2025]. A second independent parameter, the NTK-task alignment, determines whether the lazy solution generalises: if the initial NTK is well-aligned with the task, the lazy regime itself generalises and no transition (and hence no grokking) is necessary.

2.3 The Learned Algorithm: From Fourier Modes to Group Representations

The GCR algorithm. Chughtai et al. [2023] generalise Nanda et al.'s Fourier multiplication algorithm from cyclic groups to arbitrary finite groups via the *Group Composition via Representations* (GCR) algorithm. For any finite group G with irreducible representations $\rho_1, \ldots, \rho_k$, the algorithm proceeds: (1) the embedding layer maps group elements a, b to representation matrices $\rho(a), \rho(b)$; (2) the MLP layer uses ReLU activations to compute $\rho(a)\rho(b) = \rho(ab)$; (3) the unembedding reads off logits via characters $\chi_\rho(abc^{-1}) = \mathrm{tr}(\rho(abc^{-1}))$, which are maximised when $c = ab$. The Fourier algorithm of Nanda et al. [2023] is literally a special case: each Fourier frequency ω_k corresponds to an irreducible 2D rotation representation of the cyclic group C_p, and the logit $\cos(2\pi\omega_k(a + b - c)/p) = \frac{1}{2}\chi_{\rho_k}(abc^{-1})$.

Chughtai et al. [2023] find evidence for *weak but not strong universality*: all models use variants of GCR, but the specific representations chosen and the order they develop vary across random seeds and architectures. Notably, they observe multiple phases of grokking (their Figure 14), with different representations locking in at different times.

The coset circuit alternative. Stander et al. [2024] challenge the GCR interpretation. Studying the same networks trained on S_5 and S_6, they argue that models implement *coset circuits*: each neuron specialises in recognising whether the product $\sigma_l\sigma_r$ falls in a specific coset of a specific subgroup, rather than computing matrix products of representations. The model discovers the true subgroup structure of the group. The embeddings encode coset membership (not representation matrices), and the linear layer combines this information via the predictable multiplication properties of conjugate subgroups' cosets.

The tension with GCR arises because coset membership functions correlate with characters of irreducible representations (Stander et al., Appendix E), so Fourier-space evidence is consistent with both interpretations. The disagreement is about what the neurons *actually compute*: matrix multiplication of representations (GCR) vs. coset membership testing (cosets).

The ρ-set unification. Wu et al. [2024] partially unify these accounts by identifying *ρ-set circuits*: within each irreducible representation ρ, the key vectors organising the neurons form a ρ-set, an orbit of G acting on vectors via ρ. This is more structured than GCR (which leaves the relationship between neurons within a representation unconstrained) and connects to the coset interpretation (ρ-sets of the standard representation of S_n correspond to cosets of S_{n-1}). They also show (Appendix F) that the causal interventions used by Stander et al. [2024] are insufficient to distinguish the algorithms: the same intervention results hold for $\mathbb{Z}/53\mathbb{Z}$, which has no non-trivial subgroups. Their most significant methodological contribution is *compact proofs*: translating the mechanistic interpretation into a formal verifier that guarantees model accuracy faster than brute-force evaluation, providing quantitative verification that informal ablations cannot.

Implications for this project. This ongoing debate, three papers reverse-engineering the same networks with partially conflicting conclusions, directly motivates the belief geometry approach. Belief geometry does not depend on how one interprets weights; it depends only on the input-output map and intermediate representations. If the model is implementing coset circuits, the belief geometry at the hidden layer should show clustering by coset membership (discrete, combinatorial structure). If it is implementing GCR, the geometry should show continuous representation-theoretic structure (matrix elements of representations). These are distinguishable geometries, and the computational mechanics framework provides a principled way to read them off.

2.4 Computational Mechanics Perspective

Poncini [2025] introduces two predictively-equivalent generative processes for modular addition. These are not competing data-generating processes; they are two coordinate-system presentations of the *same* conditional distribution $P(\text{future} \mid \text{history})$, related by the discrete Fourier transform on $C_p = \mathbb{Z}/p\mathbb{Z}$. In representation-theoretic terms, the simplex / HMM basis is the standard basis of the regular representation ρ_{reg} of C_p, while the polygon / EHMM basis decomposes ρ_{reg} into its irreducible characters (or a subset thereof).

RRMod$_p$ (HMM). The exact process, whose transition matrices $T_p^{(x)}$ are defined via the permutation representation ρ_σ of the cyclic group C_p. The predictive vectors lie on the vertices of a $(p-1)$-simplex $\mathcal{S}_p = \{\langle e_p^{(i)}| : i = 0, \ldots, p-1\}$. This is the regular-representation presentation: each indicator $\langle e_p^{(c)}|$ corresponds to a standard basis vector of ρ_{reg}.

sRRMod$_p$ (EHMM). An approximate process whose transition matrices $H_p^{(x)}$ are defined via 2D rotation representations $\rho_\gamma^{(\omega)}$ of C_p at frequencies $\omega_1, \ldots, \omega_n$. The predictive vectors lie on the vertices of n distinct p-gons. This is a strict subrepresentation of ρ_{reg}: the direct sum of n out of p irreducible characters. The Dirichlet identity expresses each simplex vertex as the complete Fourier sum over all p characters; the polygon target is the partial sum over $\{\omega_1, \ldots, \omega_n\}$. The logits take the form

$$z(abc) = \sum_{k=1}^{n} \frac{\alpha_k^2}{p^2} \cos\left(\frac{2\pi\omega_k(a+b-c)}{p}\right). \tag{1}$$

The key empirical finding of Poncini [2025] is that the trained transformer's activations are linearly aligned with the EHMM presentation cleanly but only weakly with the HMM-simplex presentation ($R^2 = 0.99$ vs. $R^2 = 0.09$). Because the two are predictively equivalent, this is not the model "choosing" between competing DGPs; it is a statement about which coordinate system the model's activations are naturally organised in. A research question this framing opens (taken up in RQ1) is what determines the simplex R^2: in particular, whether the gap between EHMM and simplex decodability reflects how many irreducible characters the model has committed to (the partial Fourier sum being a closer or worse approximation to the full Dirichlet identity), and whether different architectures land at different points on this concentration spectrum. The residual stream decomposition is:

Position	**Decoded predictive vector**
Post-embedding, position a	$\langle v^{(a)}\|$
Post-embedding, position b	$\langle v^{(b)}\|$
Post-attention, position b	$\langle v^{(a)}\| + \langle v^{(b)}\|$
Post-MLP, position b	$\langle v^{(a+b \bmod p)}\|$

2.5 Singular Learning Theory and the Local Learning Coefficient

In singular learning theory (SLT) [Watanabe, 2009], the free energy of a statistical model admits the asymptotic expansion

$$F_n = nL_n(w^*) + \lambda \ln n - (m-1) \ln \ln n + O(1), \tag{2}$$

where λ is the real log canonical threshold (RLCT) and m is the multiplicity. The RLCT measures the effective complexity of the model in the neighbourhood of the current solution: lower λ corresponds to a simpler, more degenerate region of parameter space.

The local learning coefficient (LLC) is a local version of the RLCT. We are more interested in this quantity because we want to obtain information about the geometry around a local minimum during training. We have an estimator of λ [**?**], computed via SGLD sampling around a given checkpoint. In developmental interpretability [Hoogland et al., 2024], the LLC is tracked throughout training to identify phase transitions in the model's effective complexity.

RLCT of deep linear networks. Pepin Lehalleur and Rimányi [2025] recently showed that for deep linear networks (DLNs) with loss $K_B^{\mathrm{DLN}}(A_*) = \|\mathrm{mult}(A) - B\|_2^2$, the RLCT takes the clean form

$$\mathrm{rlct}(K_B^{\mathrm{DLN}}) = \frac{\mathrm{codim}\,\mathrm{mult}^{-1}(B)}{2}, \tag{3}$$

meaning DLNs are "mildly singular", the RLCT saturates the upper bound $\mathrm{rlct}(F) \leq \mathrm{codim}\, F^{-1}(0)/2$ that holds for general real analytic functions. This result, which reformulates and clarifies earlier work by Aoyagi [2024], provides an exact baseline for our analytical RLCT programme: the gap between the linear RLCT and the actual nonlinear RLCT for the two-layer MLP measures the contribution of the nonlinearity to the singularity structure. They also establish a surprising permutation invariance: the codimension (and hence the RLCT) depends on the *multiset* of layer widths, not their ordering.

Compression-based complexity. DeMoss et al. [2025] independently track the complexity dynamics of grokking using a measure based on rate-distortion theory and Kolmogorov complexity. Their approach coarse-grains network weights via low-rank decomposition and quantisation, then measures the compressed description length. They observe the same rise-then-fall complexity pattern we predict for the LLC: complexity increases during memorisation and falls sharply during generalisation, while unregularised networks remain trapped at high complexity indefinitely. Their measure is deterministic and reproducible, complementing the stochastic SGLD-based LLC estimation, and we adopt it as a triangulation tool (see section 4.4).

2.6 Data Attribution via Unrolling

The unrolling framework [Bae et al., 2024, Jaburi, 2026] treats training as a composition of differentiable update maps and computes the sensitivity of final-model observables to perturbations of individual training example weights:

$$\left.\frac{\partial w_T(\beta)}{\partial \beta_i}\right|_{\beta=\mathbf{1}} = -\sum_{t=0}^{T-1} \frac{\eta_t}{n}\, \mathbf{1}[i \in B_t]\, J_{(t+1):T}\, \nabla_w L(w_t, z_i), \tag{4}$$

where $J_{(t+1):T} = \prod_{s=t+1}^{T-1} J_s$ is the ordered product of step Jacobians $J_t = I - \eta_t \hat{H}_t$. For full-batch GD, the indicator $\mathbf{1}[i \in B_t] = 1$ for all i and t, and the formula is deterministic.

2.7 Dynamical Systems Framework

Under full-batch gradient descent (which we use throughout to eliminate stochastic batch noise), training is a deterministic dynamical system: the parameter update map $f(w) = w - \eta\nabla L(w) - \eta\kappa w$ defines an iterated map on $\mathbb{R}^P$ whose orbits are the training trajectories. This subsection fixes the dynamical-systems vocabulary the methodology of section 4.7 relies on, and locates the other three frameworks within it.

Fixed points, attractors, basins. A fixed point w^* of f satisfies $\nabla L(w^*) + \kappa w^* = 0$. It is (locally) *stable* if the linearised map $J_* = I - \eta\hat{H}(w^*) - \eta\kappa I$ has all eigenvalues inside the unit circle, where $\hat{H} = \nabla^2 L$ is the loss Hessian; such fixed points are *attractors*, with associated *basins of attraction* $\mathcal{B}(w^*)$ comprising the initial conditions whose trajectories converge to them. Our central hypothesis is that the lazy/memorisation solution and the rich/generalisation solution are two distinct attractors of the same dynamical system on the same task, with their relative stability and basin sizes controlled by the effective laziness L. Grokking corresponds to a trajectory crossing the basin boundary: a long sojourn near the lazy attractor before being kicked into the rich attractor's basin.

Bifurcations. A *bifurcation* is a qualitative change in attractor structure as a control parameter varies. The classical types relevant here, and their characteristic delay scalings as a control parameter L approaches a critical value L^*, are:

- *saddle-node*: a stable and an unstable fixed point collide and annihilate, with $\tau \sim |L - L^*|^{-1/2}$;
- *transcritical*: two fixed points exchange stability, with $\tau \sim -\log|L - L^*|$;
- *pitchfork*: typically associated with broken symmetry, with one stable fixed point splitting into two stable and one unstable.

The bifurcation type can in principle be read off empirically from how the grokking delay τ scales as $L \to L^*$, providing direct evidence about the mechanism (section 4.7).

Stability via the Jacobian eigenspectrum. The eigenspectrum of the linearised step-update map $J_t = I - \eta_t\hat{H}_t$ at each training checkpoint tracks how close the dynamics are to instability. Eigenvalues approaching the unit circle indicate emerging slow-manifold directions along which the trajectory may eventually depart from the current attractor. In grokking, the prediction is that the lazy attractor gradually loses stability along feature-learning directions during the apparent memorisation plateau, well before the visible test-accuracy transition.

Relation to the other frameworks. The LLC (section 2.5) characterises the local effective dimensionality of each attractor, a complexity-flavoured description of the attractor itself, orthogonal to its location or stability. Belief geometry (section 2.4) characterises the *algorithmic content* of each attractor: what the model represents once it has converged there, with the lazy and rich attractors having qualitatively different belief-geometry signatures (unstructured vs. structured). Unrolling-based data attribution (section 2.6) decomposes the trajectory between attractors by attributing each parameter step to specific training examples, identifying which examples drive the basin transition. The dynamical-systems perspective identifies *what the transition is* and *what governs it*; the other three frameworks provide complementary characterisations of the attractors and the trajectory between them.

Note on stochastic vs. deterministic training. Full-batch GD gives a deterministic map and so admits the classical fixed-point / basin / bifurcation analysis cleanly. Stochastic mini-batch SGD makes training a stochastic dynamical system whose long-time behaviour is described by an invariant measure rather than a fixed point; many of the same qualitative concepts (metastable states corresponding to attractors of the noiseless flow, escape times, basin boundaries) still apply but require stochastic-dynamics machinery to make rigorous. The choice of full-batch GD throughout this project is methodologically motivated by this distinction: the deterministic setting is where the dynamical-systems vocabulary applies most cleanly and where the bifurcation analysis is sharpest.

3 Research Questions

RQ1. Can belief geometry identify learned algorithms model-agnostically? Applying the computational mechanics framework to the three architectures of section 2.1 on modular addition, what does the belief geometry of each reveal about the algorithm it implements? We have three parts: (a) does each architecture's geometry indicate membership of a shared algorithmic family (Fourier multiplication / GCR [Chughtai et al., 2023])? (b) where geometries differ, do the differences reflect distinct algorithms or different parameterisations of the same family? (c) which diagnostics in the pipeline are sensitive to algorithm-family vs. parameterisation, and can we read these axes off independently?

A further dimension is the dependency on the DGP. The Phase 1 pipeline depends on the DGP only to construct the predictive-vector targets $\langle v^{(x)}|$; every other diagnostic is DGP-agnostic. Two routes can in principle remove this remaining dependency: Crutchfield-style empirical causal-state reconstruction [Shalizi and Klinkner, 2004] (an empirical V from data alone), and candidate-family comparison (positing a library of plausible algorithmic structures and selecting via comparative R^2). The three-architecture design (section 2.1) operationalises this: the calibration tier (transformer + Chughtai MLP) cross-validates against ground truth; the Kumar MLP on modular addition is the same-task test; the Kumar MLP on polynomial regression is the cross-task extension.

Subsidiary questions: Can the belief geometry of the MLP reverse-engineer the algorithm where direct weight analysis is not available? Does it decompose by model component, revealing which is responsible for which part of the algorithm? Can it resolve the GCR vs. coset circuit debate [Chughtai et al., 2023, Stander et al., 2024, Wu et al., 2024] by distinguishing continuous representation-theoretic structure from discrete coset clustering at the hidden layer (most acute for non-abelian groups, where the cyclic-group instantiation does not suffice to discriminate)?

RQ2. What is the complexity profile of grokking? How does the LLC evolve throughout training, and does the grokking transition correspond to a sharp drop in the LLC? Is this drop universal across different mechanisms for triggering grokking (weight decay, output scale α, λ-balancedness)? How does the LLC decompose across model components (embedding, attention, MLP, unembedding)?

RQ3. What is the representational trajectory of grokking? How does the belief state geometry (the EHMM predictive vectors) form during training? In what order do the different components of the algorithm develop, and does this ordering correlate with the component-wise LLC transitions?

RQ4. What drives the transition causally? Which training examples are most influential at each stage of grokking, and through which model components do they act? Does the data attribution pattern shift qualitatively from diffuse (memorisation/lazy-type) to structured (group-theoretic) during the transition?

RQ5. What is the dynamical mechanism? Can grokking be characterised as a transition from a lazy regime (fixed NTK, high LLC, unstructured belief geometry) to a rich regime (evolving NTK, low LLC, structured belief geometry)? Is the memorisation phase the lazy regime, and is the generalisation phase the rich regime?

RQ6. What is the bifurcation parameter? Is there an effective laziness parameter (a function of weight decay κ, output scale α, initialisation scale σ_{init}, and λ-balancedness) that governs the bifurcation? Does the grokking delay collapse onto a single curve when plotted against this effective parameter? What is the role of NTK-task alignment as a second, independent axis?

RQ7. How do causal attribution and complexity reduction interact? Do the natural data partitions identified by the unrolling attribution (clusters of examples with correlated influence) predict the subsets for which the data-refined LLC drops earliest? Can the unrolling be used to discover, rather than assume, the data partitions most relevant to the complexity transition? For example, if we identify a subset of data which (by the LOO naive causality assumption) has the most significant influence, at what point does swapping datapoints from this set with data points with insignificant influence result in the model 'ungrokking'? (This is related to a hypothesis described in [Varma et al., 2023] where decreasing the size of the training set was thought to lead to ungrokking).

RQ8. Does SGD in singular models have an intrinsic simplicity bias? Is the lazy-to-rich transition evidence that SGD is biased toward low-LLC solutions? Does the LLC reliably decrease during training across all mechanisms that induce feature learning, even when parameter norm increases?

4 Methodology

4.1 Experimental Setup

All experiments use full-batch gradient descent (eliminating stochastic batch noise) to obtain a deterministic dynamical system. For the transformer, the architecture and hyperparameters follow Poncini [2025]: single-layer single-head transformer, $d_{\text{model}} = 128$, $d_{\text{mlp}} = 512$, $p = 113$, AdamW with learning rate 10^{-3} and weight decay 1.0, trained for 25,000 epochs. For the two-layer MLP, the setup follows Kumar et al. [2024] with controlled output scale α. Checkpoints are saved every 10–50 epochs.

4.2 Step 1: Belief Geometry as Algorithm Identification

We apply the computational mechanics framework across the three architectures of section 2.1: the transformer and the Chughtai-style MLP at the calibration tier (where the learned algorithm has independent ground truth from prior reverse-engineering), and the Kumar-style MLP at the bootstrap-test tier (first on modular addition, then more ambitiously on polynomial regression). The calibration-tier architectures are analysed jointly throughout this step; the Kumar architecture is brought in as a follow-on with the same pipeline but the additional goal of recovering algorithm-family identification without prior reverse-engineering to lean on.

Decoding probes. For each chosen activation position, we use two complementary linear-decoding probes:

(i) **Full-dimensional supervised ridge.** Regress activations $X \in \mathbb{R}^{p^2 \times d}$ onto a target Y (an EHMM polygon target or the HMM simplex control) using ridge regression on all d activation dimensions. This tests the linear-representation hypothesis directly: "is the predictive vector linearly recoverable from the activation at all?"

(ii) **PCA-restricted ridge.** Reduce activations to their top K principal components, then regress these onto the target. This is the recipe of Poncini [2025] and tests the strictly stronger claim: "is the predictive-vector representation concentrated in the top-K variance directions?" For representations that commit to few high-amplitude modes, this works cleanly with small K; for representations that spread amplitude across many modes of similar weight, K must be at least $2n_{\text{modes}}$ and the probe may still fail to isolate individual modes.

The pair of probes is itself diagnostic: the gap between them measures representational concentration in the top-K PCs, a quantity that varies meaningfully across architectures. Where this section refers to "decoding R^2" below, the primary probe is the full-dim supervised version unless otherwise stated.

Transformer belief geometry. At each checkpoint, decode the sRRMod$_p$ predictive vectors from model activations at each residual stream position identified in Poncini [2025]:

(a) **Post-embedding pre-attention (positions a and b):** Fit activations to $\langle v^{(a)}|$ and $\langle v^{(b)}|$. Track R^2 and visualise the circle formation at each frequency ω_k.

(b) **Post-attention pre-MLP (position b):** Fit activations to $\langle v^{(a)}| + \langle v^{(b)}|$. Track when the additive structure appears.

(c) **Post-MLP pre-unembedding (position b):** Fit activations to $\langle v^{(a+b \bmod p)}|$. Track when the modular sum representation forms.

At each checkpoint, we also fit to the HMM simplex predictive vectors $\langle e_p^{(ab)}|$ as a control. Per the reframing in section 2.4, the simplex R^2 is interpreted as a Fourier-coverage diagnostic (how close the encoded subrepresentation of ρ_{reg} is to the full regular representation) rather than as a yes/no test for "representing the HMM".

MLP belief geometry. Apply the same decoding pipeline as for the transformer. Extract activations at each architectural position of interest, pre-nonlinearity ($h = W_a[a] + W_b[b]$ for the Chughtai variant, $h = W_1 x$ for the Kumar variant) and post-nonlinearity ($\sigma(h)$), and fit them to position-appropriate EHMM and HMM-simplex targets via the dual probes above. The position-appropriate EHMM target depends on the layer: the pre-nonlinearity layer's natural target is the additive $\langle v^{(a)}| + \langle v^{(b)}|$ (the analogue of the transformer's post-attention layer); the post-nonlinearity layer's natural target is $\langle v^{(a+b \bmod p)}|$ (the analogue of the transformer's post-MLP layer). At the post-nonlinearity layer, a methodological refinement is to also fit the regression on *class-mean activations* (averaging activations over inputs sharing a value of $a + b \bmod p$), which removes per-(a, b) cross-term variance from the regression target.

Per-frequency analysis. For each individual fitted frequency ω_k, fit the activation to the 2-D target $(\cos 2\pi\omega_k c/p, \sin 2\pi\omega_k c/p)$ (or the appropriate sum-of-circles target at additive positions) via the dual probes. Report the per-frequency R^2 values as a heatmap across positions and frequencies. Visualise the model's predicted points alongside the theoretical target as a per-frequency pair of 2-D scatter plots.

Cross-architecture comparison. The question "do the two architectures have the same belief geometry?" admits more cases than a simple match/mismatch dichotomy. Following section 2.4, both architectures (if they implement any Fourier-style algorithm at all) encode some subrepresentation of ρ_{reg}, and they may differ in (i) which subrepresentation, (ii) how concentrated the α^2 amplitudes are, (iii) how variance is distributed across activation directions, or (iv) the algorithmic family itself. We therefore anticipate four cases when comparing the converged belief geometries:

(a) **Same algorithm and same parameterisation:** both architectures encode the same subrepresentation with the same concentration profile. Diagnostics across all probes agree.

(b) **Same algorithm, different parameterisation:** both architectures encode subrepresentations of ρ_{reg} but with different cardinality $|\Omega|$ and/or different α^2 concentration. Full-dim decoding R^2 for the EHMM target may pass cleanly in both, while PCA-restricted decoding may pass in one and not the other; simplex R^2 may differ substantially as a function of $|\Omega|$. This is the case where architectures share an algorithmic family (Fourier multiplication / GCR [Chughtai et al., 2023]) but parameterise it at different points on a concentration $\leftrightarrow$ redundancy spectrum.

(c) **Different algorithms (both structured):** the architectures encode genuinely different non-equivalent structures. Belief-geometry signatures differ at the algorithmic-family level, not just parameterisation. The LLC at convergence then becomes the relevant tie-breaker for which algorithm is "simpler" in the SLT sense.

(d) **One structured, one unstructured:** the architectures differ in their capacity to learn structured solutions for this task.

A key methodological question is which of our probes are sensitive to which axis. Full-dim supervised ridge against the EHMM target is expected to be sensitive to algorithm-family but not to parameterisation within the family; PCA-restricted ridge is sensitive to representational concentration; simplex R^2 via the Dirichlet identity is sensitive to subrepresentation cardinality. Reading these axes off independently (distinguishing algorithm-level from parameterisation-level differences) is the methodological contribution required to claim that belief geometry is a model-agnostic algorithm-identification tool.

Resolving the GCR vs. coset debate. The GCR/coset controversy (section 2.3) provides a concrete test case for belief geometry's discriminating power. At the hidden layer of models trained on S_5, if the model implements GCR, the intermediate predictive states should show continuous representation-theoretic structure (matrix elements of $\rho(ab)$ organising smoothly within each irreducible representation). If it implements coset circuits, the hidden-layer geometry should show discrete clustering by coset membership, with neurons specialising in specific subgroup-coset pairs. The ρ-set interpretation of Wu et al. [2024] predicts an intermediate structure: orbits under the group action via ρ, which are discrete but representation-theoretically organised.

Extension to non-abelian groups. To strengthen the test, we extend the belief geometry analysis to models trained on S_5 multiplication (following the setup of Chughtai et al. [2023], Stander et al. [2024], Wu et al. [2024]). For non-abelian groups, the algorithm space is richer: the group has multiple non-isomorphic irreducible representations of varying dimension, and the GCR/coset ambiguity is most acute. The belief geometry for S_5 should reflect the ρ-set structure: orbits under irreducible representations, with the specific representations selected varying across seeds (weak universality). Comparing the belief geometry of the MLP and transformer trained on S_5 would be a particularly strong test of model-agnostic algorithm identification.

Compact proofs as verification. Following Wu et al. [2024], if belief geometry identifies the algorithm, this identification can in principle be formalised as a compact proof of model accuracy. Construct idealised parameters from the belief geometry description and verify that the idealised model achieves high accuracy faster than brute-force evaluation. This provides quantitative verification stronger than R^2 fits or ablations alone.

Per-frequency decomposition. Track the R^2 of the fit to each individual circle component at frequency ω_k (or more generally, each irreducible representation ρ) separately, to determine whether different representations lock in at different times during the grokking transition. Chughtai et al. [2023] observe that different representations emerge at different training stages, sometimes producing multiple phases of grokking; this predicts that the LLC curve may show multiple steps corresponding to sequential representation learning.

4.3 Step 2: LLC Tracking During Training

At each checkpoint, we estimate the LLC via SGLD sampling around the current parameters, using the `devinterp` library.

Full LLC. SGLD with full-batch gradient evaluation (feasible since $p^2 = 12{,}769$ examples). Multiple independent chains with $\hat{R}$ convergence diagnostics. Hyperparameter sweeps over step size, localisation strength γ, and chain length to verify stability of the LLC estimate.

Component-wise LLC. Freeze all model components except one (embedding, attention, MLP, or unembedding) and estimate the LLC over the free component's parameters only. This decomposes the global complexity transition into per-component contributions and reveals whether components undergo their transitions simultaneously or sequentially.

Representation-refined LLC. Further decompose within components: for the embedding, estimate the LLC over the subspace corresponding to each learned frequency ω_k (or irreducible representation ρ); for the attention, separate the QK circuit ($W_Q^\top W_K$) from the OV circuit ($W_O W_V$). The number of learned frequencies is itself an empirical quantity that varies across architectures and seeds, and the representation-refined LLC must adapt to whatever cardinality the belief-geometry analysis (Step 1) returns: a model that commits to a small subrepresentation of ρ_{reg} contributes fewer per-frequency LLC components than one that commits to a larger subrepresentation. Since Chughtai et al. [2023] observe that different representations lock in at different training stages, sometimes producing multiple distinct phases of grokking, the representation-refined LLC should show sequential drops corresponding to the sequential emergence of each representation circuit. This connects the developmental interpretability perspective (when does complexity change?) to the mechanistic interpretability perspective (which circuit is forming?).

Data-refined LLC. Estimate the LLC using subsets of the training data to determine whether the complexity transition occurs uniformly across the data or proceeds in a structured fashion. The choice of data partition is itself a methodological question: one could partition a priori by algebraic structure (e.g., by the value of $a + b \bmod p$, or by a alone), but the model may organise the data along axes that do not correspond to any single obvious algebraic quantity. We address this by using the unrolling-based attribution (Step 3) to discover the natural partitions empirically, as described in section 4.6.

Universality across mechanisms. A key prediction is that the LLC drop should occur regardless of which mechanism triggers the lazy-to-rich transition. We track LLC curves across sweeps of weight decay κ, output scale α, λ-balancedness, and spectral entropy regularisation [DeMoss et al., 2025]. The spectral entropy regulariser $\mathcal{L}_{\text{reg}}(\theta) = \beta H_{\text{svd}}(\theta)$ penalises the effective rank of weight matrices directly, providing a qualitatively different mechanism from weight decay or output scaling: it targets complexity rather than indirectly forcing feature learning. If the LLC drop is universal even under this regulariser, that strengthens the case for universality.

4.4 Step 2b: Compression-Based Complexity Triangulation

At each checkpoint where we compute the LLC, we also compute the lossy compression complexity of DeMoss et al. [2025] (Algorithm 1 of their paper: low-rank decomposition + quantisation + lossless compression via bzip2). This provides a deterministic, reproducible complexity measure that does not rely on SGLD convergence. If both measures track the grokking transition consistently (rising during memorisation, falling during generalisation), this provides independent validation of each and addresses the methodological concern about SGLD reliability. Disagreement would be informative about what each measure is actually capturing: the LLC reflects loss landscape singularity structure, while the compression measure reflects weight-space compressibility. The relationship between these connects to the MDL-SLT correspondence established in Hoogland et al. [2025].

4.5 Step 3: Data Attribution via Unrolling

Using the unrolling framework, compute the influence of each training example $z_i = (a, b, a{+}b \bmod p)$ on the test loss at each training step.

Since training uses full-batch GD, the unrolling formula (4) is deterministic and exact. For computational feasibility, we use the SOURCE approximation [Bae et al., 2024]: divide training into L segments at checkpoints, apply the stationarity approximation within each segment, and use the matrix exponential approximation for segment Jacobians. The resulting filter

$$F_r(\sigma) = \frac{1 - e^{-\bar{\eta} K \sigma}}{\sigma} \tag{5}$$

provides principled damping determined by the training process itself (learning rate $\bar{\eta}$, segment length K, Hessian eigenvalue σ).

Attribution structure analysis. At each stage of training, examine whether the attribution pattern is (a) diffuse, with each example contributing roughly equally, consistent with the lazy/memorisation regime, or (b) structured, where examples sharing algebraic structure (e.g., the same value of $a + b \bmod p$) are correlated in their influence, consistent with the model exploiting group structure in the rich regime.

Component-wise attribution. Decompose the Jacobians into per-component contributions to track which training examples drive changes in which model components at each stage. The examples most influential on the embedding should be those that help organise the circles; the examples most influential on the MLP should be those that help learn the modular sum.

Attribution clustering. At each checkpoint, collect the per-example attribution vectors into a matrix (rows indexed by training examples, columns by parameter directions). Cluster this matrix (e.g., via spectral clustering or k-means on the attribution vectors) to identify groups of training examples that exert correlated influence on the model. During the lazy/memorisation phase, we expect these clusters to be unstructured; as the model transitions to the rich/generalisation regime, we expect clusters to align with the algebraic structure of modular addition. The evolution of this clustering provides a data-driven decomposition of the training set that reflects the model's internal organisation at each point in training.

4.6 Step 4: Synthesis via Unrolling-Informed Data-Refined LLC

We will use the unrolling-based attribution to inform the data-refined LLC analysis. These two frameworks are connected at a fundamental level: the unrolling propagates per-example influence through the step Jacobians $J_t = I - \eta \hat{H}_t$, and the LLC is determined by the Hessian

eigenspectrum that defines these same Jacobians. The two frameworks are reading off different aspects of the same curvature structure: the LLC integrates over it to produce a scalar complexity measure, while the unrolling uses it to propagate per-example influence vectors.

This connection motivates the following workflow at each checkpoint:

(i) **Compute unrolling attributions** for all training examples, obtaining a matrix of per-example attribution vectors.

(ii) **Cluster the attribution vectors** to identify natural data partitions: groups of examples that the model treats similarly at this point in training. These clusters need not correspond to any a priori algebraic partition; they reflect the model's internal organisation.

(iii) **Compute the data-refined LLC** for each identified cluster. This measures the effective complexity of the model with respect to each group of examples separately.

(iv) **Track the co-evolution** of the attribution structure and the per-cluster LLC. The prediction is that clusters whose attribution pattern shifts from diffuse to group-theoretically structured are precisely the clusters whose data-refined LLC drops.

If this correspondence holds, it establishes a direct link between the causal drivers of the grokking transition (which examples are pushing the model toward the rich regime) and the complexity reduction that constitutes the transition (which subsets of the loss landscape are becoming simpler). This would provide a unified causal-complexity account: specific training examples drive the model toward a region of parameter space where the loss landscape, restricted to the data those examples represent, has lower effective dimension.

4.7 Step 5: Dynamical Systems Analysis and Bifurcation

We treat training as a deterministic dynamical system $f : \mathbb{R}^d \to \mathbb{R}^d$ (the full-batch update map) and characterise the basin structure, the lazy-to-rich transition, and the bifurcation.

Order of application across architectures. The dynamical-systems analysis is applied to the calibration-tier architectures (transformer and Chughtai MLP) first, where the belief-geometry ground truth gives an independent characterisation of what the rich attractor's algorithmic content should look like and the LLC drop can be cross-validated against the formation of the known Fourier basis. Once the methodology is anchored on the calibration tier, the same dynamical-systems analysis is applied to the Kumar architecture (modular addition, and subsequently polynomial regression if stage (i) of section 2.1 succeeds). To make the sweep over the output-scale parameter α uniform across architectures, we use the architectural addition described in section 2.1: an explicit α factor on the unembedding output of the transformer and the Chughtai MLP, matching the explicit α already present in the Kumar architecture.

Identifying (or refuting) the effective laziness parameter. The working hypothesis is that the bifurcation governing grokking is parameterised, at leading order, by an *effective laziness* $L(\kappa, \alpha, \sigma_{\text{init}}, \lambda)$ together with the NTK-task alignment A, rather than by each of these hyperparameters independently. The methodology in this subsection is structured both to identify such an effective parameter if one exists and to detect its failure if the dimension reduction does not hold (e.g., if σ_{init} acts as a third independent axis rather than collapsing onto L together with κ and α).

To identify L empirically, we run a 2D sweep over output scale α and weight decay κ, recording the grokking delay τ (epochs until test accuracy exceeds a threshold, or ∞ if grokking does not occur). The level curves of τ in the (α, κ) plane reveal the functional form of L: if the level curves are straight lines with slope s in log-log space, then $L \propto \kappa/\alpha^s$. Once a candidate L is

identified, we verify collapse: all (α, κ) combinations should fall on a single curve $\tau(L)$. Adding initialisation scale σ_{init} and λ-balancedness as additional axes tests whether L captures the full effective laziness.

The functional form of $\tau(L)$ near the critical value L^* identifies the bifurcation type: $\tau \sim |L - L^*|^{-1/2}$ suggests a saddle-node bifurcation; $\tau \sim -\log|L - L^*|$ suggests a transcritical bifurcation.

NTK-task alignment as a second axis. The NTK-task alignment A is measured via kernel-target alignment (KTA):

$$\text{KTA} = \frac{\langle K, yy^\top \rangle_F}{\|K\|_F \|yy^\top\|_F}, \tag{6}$$

where K is the NTK matrix at initialisation. High alignment means the lazy solution generalises; low alignment means it does not. We sweep both L and A to map the bifurcation surface: the boundary of the grokking region in the (L, A) plane. The LLC should only show the characteristic drop-then-plateau in the grokking region of this plane.

Control experiment: no feature learning. Train with large α (ensuring lazy dynamics) and confirm that the model memorises and remains memorised indefinitely (stable high LLC, unstructured belief geometry, high test loss, fixed NTK). This establishes the lazy/memorisation solution as a genuine stable attractor of the lazy-regime dynamics.

Feature-learning onset experiment. Train in the lazy regime, then change a control parameter (α, weight decay, or λ-balancedness) to induce feature learning. Track the delay between onset of feature learning and onset of grokking. Measure the LLC, belief geometry, NTK kernel distance, and data attribution throughout.

Reversibility experiment. Induce feature learning, allow the LLC to begin changing, then revert the control parameter. Determine whether the model re-stabilises in the lazy/memorisation regime (the transition is reversible) or continues to the generalisation solution (a point of no return has been crossed). This maps the basin boundary along the training trajectory.

Stability analysis. At checkpoints during the lazy/memorisation phase, compute the eigenspectrum of the Jacobian of the update map $J_t = I - \eta \hat{H}_t$. Track how the spectral radius evolves as the effective laziness decreases. The hypothesis predicts that eigenvalues should approach or cross the unit circle as the lazy regime is destabilised.

Basin mapping. Perturb the fully grokked model's weights and retrain from each perturbation to map the generalisation basin. The generalisation basin is predicted to be small relative to the memorisation/lazy basin, explaining why random initialisation essentially always produces memorisation first.

λ-balanced initialisation experiment. Following Dominé et al. [2025], initialise the transformer with controlled λ-balanced weights and sweep λ from 0 to large positive/negative values. For each λ, record: grokking delay, LLC curve, belief geometry evolution, NTK kernel distance from initialisation. The prediction is that small $|\lambda| \to$ short/no lazy phase $\to$ early LLC drop $\to$ early polygon formation $\to$ small/no grokking delay $\to$ large NTK movement.

5 Expected Outcomes

If the lazy-to-rich transition hypothesis is correct, the combined measurements should support a narrative of the following form, with each claim backed by specific experimental evidence:

1. **The memorisation solution is a high-complexity lazy attractor.** In the lazy regime (large α, large $|\lambda|$, no weight decay), the model converges to a high-LLC solution with unstructured belief geometry, fixed NTK, and remains there indefinitely. The lazy/memorisation basin is large (most random initialisations lead to it).

2. **Feature learning destabilises the lazy regime.** Any mechanism that induces feature learning (weight decay, small α, balanced initialisation) causes the NTK to evolve and the LLC to change, reflecting the model being pushed out of the lazy regime. The Jacobian eigenspectrum confirms increasing instability.

3. **At a critical point, the model undergoes a phase transition.** The LLC drops sharply. The belief geometry snaps into the polygonal structure predicted by sRRMod_p. The NTK kernel distance from initialisation increases rapidly. The data attribution pattern shifts from diffuse to group-theoretically structured.

4. **The transition has internal structure.** The component-wise LLC and belief geometry tracking reveal a specific ordering: embeddings organise into circles, then attention learns the additive structure, then the MLP learns the modular sum. Each sub-transition corresponds to a feature in the LLC curve and is driven by specific training examples.

5. **The generalisation solution is a low-complexity stable attractor.** After grokking, the LLC stabilises at a lower value. The belief geometry matches the sRRMod_p predictive vectors. The solution is robust to perturbation (large basin, though much smaller than the lazy basin at initialisation).

6. **The transition is governed by a low-dimensional bifurcation in effective control parameters.** If the dim-reduction hypothesis holds, grokking delay collapses onto a single curve when plotted against the effective laziness parameter L, regardless of which individual hyperparameter $(\kappa, \alpha, \sigma_{\text{init}}, \lambda)$ is varied; the delay diverges at a critical L^*, identifying the bifurcation; and the NTK-task alignment A determines the boundary of the grokking region in the (L, A) plane. If the dim reduction does not hold cleanly (if some of the hyperparameters do not collapse onto L, or if a third independent axis is needed), the same experimental design detects this and the effective dimension is itself the empirical answer rather than a fixed assumption.

7. **The LLC tracks the transition universally.** The LLC drops during the lazy-to-rich transition regardless of which mechanism triggers it: weight decay, output scale α, λ-balancedness, or initialisation scale. This establishes the LLC as a universal diagnostic for this transition, not tied to any particular regulariser, and provides evidence that SGD in singular models is biased toward low-LLC solutions.

8. **Belief geometry identifies the learned algorithm model-agnostically.** The computational mechanics framework reveals the algorithm learned by both the transformer and the MLP. Where the architectures' belief-geometry signatures agree, this identifies them as instantiating the same algorithmic family; where the signatures differ, the diagnostic refinements of section 4.2 (full-dim vs. PCA-restricted decoding, simplex R^2 via the Dirichlet identity, per-frequency analysis) distinguish whether the difference reflects a distinct algorithm or a different parameterisation of the same family. The expected positive outcome is that belief geometry serves as a tool capable of separating algorithm-level from

parameterisation-level differences across architectures, demonstrating its applicability in settings where direct weight analysis cannot read the algorithm out.

9. **Data attribution and complexity reduction are two views of the same process.** The natural data partitions identified by clustering the unrolling attribution vectors correspond to the subsets for which the data-refined LLC drops during grokking. Examples whose attribution shifts from diffuse to structured are precisely those whose contribution to the model's effective complexity is decreasing.

The answer to "why does grokking happen." If the central hypothesis and its dim-reduction variant are both substantiated by the experiments described above, the resulting account is the following. Grokking occurs because the model begins training in the lazy regime, where it memorises via kernel interpolation with a fixed NTK. The lazy/memorisation solution is a high-complexity attractor with a large basin (most random initialisations lead to it). Any mechanism that forces feature learning (weight decay, small output scale, balanced initialisation) destabilises this lazy regime by causing the NTK to evolve. Once feature learning begins, the model transitions to a low-complexity rich regime where it discovers the task-adapted (Fourier/group-theoretic) generalisation algorithm. The delay is the time for the effective laziness to decrease sufficiently to trigger the transition; the sharpness reflects the self-reinforcing nature of feature learning once it begins. If the dim-reduction hypothesis additionally holds, the bifurcation is governed by an effective laziness L (a function of weight decay, output scale, initialisation scale, and λ-balancedness) together with the NTK-task alignment A, rather than by each hyperparameter independently, in which case weight decay is one mechanism among several acting through a common NTK-evolution channel, and the fundamental driver is the onset of feature learning. If instead the dim reduction fails, the experimental design returns the actual effective dimension and identifies which hyperparameters act through independent channels, which would itself be a substantive empirical answer.

6 Broader Research Programme

Beyond the specific question of grokking, this project contributes to several broader research directions.

Dynamical vs. Bayesian accounts of grokking: a critical distinction. Pepin Lehalleur et al. [2025a] argue that the Bayesian posterior in singular models exhibits an internal model selection mechanism: at finite sample size n, the posterior prefers simpler solutions (lower LLC) even if they have higher loss, switching to more complex but accurate solutions as n grows. Carroll et al. [2025] provide empirical evidence for an analogy between sample size n in Bayesian inference and training steps t in SGD, suggesting that SGD trajectories exhibit a similar preference reversal.

If this analogy held generally, it would predict that the memorising solution has *lower* LLC than the generalising solution: that the model initially memorises because memorisation is simpler, and only transitions to the more complex generalising solution when enough "effective evidence" (training steps) accumulates. This is the opposite of our hypothesis, which predicts that the memorising/lazy solution has *higher* LLC (generic, unstructured, high-dimensional kernel interpolation) while the generalising solution has *lower* LLC (exploiting the low-dimensional group structure of the task). DeMoss et al. [2025] provide direct empirical support for our ordering: their compression-based complexity measure shows that complexity *rises* during memorisation and *falls* during generalisation.

Moreover, the $t \leftrightarrow n$ analogy cannot hold in general. If it did, then any model trained long enough would eventually prefer the complex generalising solution, regardless of initialisation

or laziness. But we know empirically that high-laziness models memorise forever: no amount of additional training triggers the transition. The grokking setting thus provides a concrete counterexample to the general $t \leftrightarrow n$ correspondence: the delay is explained by a *dynamical* trap (the lazy regime), not by *statistical* model selection (insufficient evidence to justify complexity).

This project can make a contribution towards adjudicate between these accounts. By measuring the LLC at both the memorising and generalising solutions, we can determine the LLC ordering directly. If the memorising solution has higher LLC (as we predict), the Bayesian model selection story does not explain grokking: the model is stuck at a *more complex* solution despite a simpler one existing, which is the opposite of what the Bayesian account predicts. This would cleanly separate the dynamical mechanism (lazy-to-rich transition, governed by effective laziness) from the statistical mechanism (internal model selection, governed by the LLC-loss tradeoff), and demonstrate that they make opposite predictions in at least one concrete setting.

SGD's simplicity bias in singular models. The above distinction does not mean SGD has no simplicity bias; it means the bias operates through the dynamics, not through Bayesian model selection. The foundational question is whether SGD in singular models has an intrinsic bias toward low-LLC solutions. If SGD consistently transitions from high-LLC to low-LLC solutions whenever feature learning is possible (and the LLC drops even when parameter norm increases, as in Kumar et al.'s no-weight-decay grokking), this is evidence that SLT complexity, not norm-based complexity, is what SGD is biased toward. But this bias is *dynamical* (SGD trajectories preferentially flow toward low-LLC regions once the lazy regime is escaped) rather than *statistical* (the posterior preferring low-LLC regions at finite n).

Belief geometry as model-agnostic algorithm identification. The most novel methodological contribution is the use of computational mechanics to identify learned algorithms in settings where direct weight analysis fails or is unavailable. "Model-agnostic" here has two distinct senses that should be distinguished, because they correspond to different stages of the broader programme and to different open problems in the field.

Architecture-agnostic. The methodology applies to any architecture that learns the task, regardless of whether the algorithm is readable from the weights. Phase 1's MLP analysis is the calibrating demonstration: GCR-style multiplication is not readable out of the weight matrices W_a, W_b, W_U as cleanly as from the transformer's matrices, yet the belief-geometry analysis recovers the same algorithmic family and characterises its parameterisation. If this works robustly across architectures (extending through deeper MLPs, multi-head attention, hierarchical compositions), it establishes belief geometry as a general tool applicable to any architecture, including those at scales where weight-level mechanistic interpretation is not feasible.

DGP-agnostic. The methodology applies even when the data-generating process itself is unknown. This is a substantially stronger claim and the main open methodological problem the broader programme would address. The Phase 1 pipeline depends on the DGP only to construct the predictive-vector targets; replacing this dependency rests on two routes: Crutchfield-style empirical causal-state reconstruction [Shalizi and Klinkner, 2004], which builds an empirical V from data alone, and candidate-family comparison, which posits a library of plausible algorithmic structures and lets comparative R^2 identify the best fit. Toy cases where the DGP is known but withheld from the analysis pipeline provide the falsifiable validation: apply the bootstrap, check whether it recovers the right answer. This last point matters: *the toy-case work calibrates the methods so that they can later be deployed when ground truth is unavailable, with the toy-case validation providing the warrant for trust at scale.*

The three-stage bootstrap programme. The natural project structure for the DGP-agnostic extension is: (i) build a catalogue of belief-geometry signatures across a library of known DGPs (cyclic groups, non-abelian groups, mixtures of cyclic groups, factored / hierarchical structures), establishing robustness to architecture and seed; (ii) given activations and data

alone, identify which DGP family in the catalogue best explains the model's belief geometry, validated on held-out toy cases where the answer is known but withheld; (iii) apply to cases where the catalogue does not contain the right DGP, demonstrating identification of structural features (symmetry group type, factor count, ergodicity, equivariance properties) without naming the specific generating process. Each stage is its own contribution, and the realistic at-scale ambition is *weak identification* (recovering the algorithmic family without naming its specific parameterisation) rather than strong identification of every learned parameter.

Mapping the project architectures to the bootstrap stages. The three-architecture design of section 2.1 delivers a partial instantiation of the programme above. The calibration tier (transformer and Chughtai MLP on modular addition) builds the first entry in the stage-(i) catalogue (cyclic-group GCR) with cross-architecture robustness checks against ground truth. The Kumar architecture on modular addition is the first stage-(ii) test: same DGP family as the catalogue entry, different architecture from those used to build it, with the methodology asked to recover the algorithm family without prior reverse-engineering to lean on. The Kumar architecture on polynomial regression is the more ambitious stage-(iii) test: a task whose DGP is not in any cyclic-group catalogue, where the methodology must identify structural features without a named generating process. Even if only the calibration tier and the first stage of the Kumar architecture are completed within the project, this constitutes proof-of-concept for the bootstrap programme; the polynomial-regression extension is the most direct test of the at-scale interpretability ambition.

Connection to current interpretability practice. The component-wise decomposition of belief geometry (tracking how the geometry changes across layers) reveals which component is responsible for which part of the algorithm, providing a form of mechanistic interpretability that complements weight-level analysis and can in principle adjudicate between conflicting weight-level interpretations. Chughtai et al. [2023] find evidence for weak but not strong universality: all models use the same *family* of algorithms (GCR), but the specific representations chosen vary across seeds. Belief geometry should detect this pattern (same geometric structure type, different specific parameters) and can potentially resolve the GCR vs. coset debate by providing a representation-independent view of what the model computes. The extension to non-abelian groups (S_5, S_6) would provide a much stronger test, since the algorithm space is richer and the interpretive ambiguity most acute.

Contribution to interpretability at scale. The interpretability programme for large language models currently lacks a principled answer to the question: "given a trained model on real-world data, what statistical structure is it actually exploiting?" The dominant activation-space tools (sparse autoencoders, probing, circuit analysis) decompose activations into features but do not directly characterise the latent structure of the data the model has internalised, and they have not so far produced the kind of structural-identification results that would answer the question above. The belief-geometry methodology developed here is a structurally different approach: it works *from* a characterisation of the data's predictive structure *into* the model's activations, rather than the other direction. If the methodology scales (via the bootstrap programme above) to settings beyond toy DGPs, it would provide an alternative interpretability route that reads out the data's latent statistical structure from the model's activations directly. This is the interpretability-and-safety angle: if we can identify what regularities a model has internalised, we can ask whether they are the regularities we intended. Belief geometry as a model-agnostic algorithm-identification tool is the contribution; the toy-case work in this project is the calibration step that grounds it.

Developmental interpretability. This project provides a detailed case study for the developmental interpretability programme, demonstrating how the LLC tracks the lazy-to-rich transition during training and how this transition corresponds to the formation of specific algorithmic structures. The component-wise and data-refined LLC analyses extend the standard

LLC methodology. The universality of the LLC signal across different mechanisms for triggering grokking would strengthen the case for LLC as a general-purpose developmental diagnostic.

Computational mechanics and the linear representation hypothesis. By tracking the belief state geometry throughout training, we connect the computational mechanics perspective (what representations the model *should* form) with the developmental perspective (when and how those representations actually form). The cross-architecture comparison (transformer vs. MLP) tests whether the same computational mechanics structure emerges in different architectures.

Analytical LLC for simple models. Related work would aim to derive analytical expressions for the RLCT of individual model components. The two-layer MLP is substantially more tractable than the transformer: the singularity structure is related to reduced-rank regression and existing results by Watanabe and Aoyagi on layered neural networks. Pepin Lehalleur and Rimányi [2025] provide the exact RLCT for deep linear networks, showing that rlct = codim/2 (the "mildly singular" result). For the nonlinear two-layer MLP, the RLCT should be *strictly lower* than this linear baseline, and the gap quantifies the effect of the nonlinearity on the singularity structure. For the transformer, the idealised Fourier logit model (1) (bypassing the transformer entirely and computing the RLCT for the logit function $\sum_k \alpha_k^2 \cos(2\pi\omega_k(a+b-c)/p))$ captures the algorithm's singularity structure and appears genuinely tractable.

References

Juhan Bae, Wu Lin, Jonathan Lorraine, and Roger Grosse. Training data attribution via approximate unrolled differentiation. *arXiv preprint arXiv:2405.12186*, 2024.

Miki Aoyagi. Consideration on the learning efficiency of multiple-layered neural networks with linear units. *Neural Networks*, 172(106132):1–11, 2024.

Liam Carroll, Jesse Hoogland, George Wang, Daniel Murfet, and Susan Wei. In-context learning and the geometry of loss landscapes. *arXiv preprint*, 2025.

Branton DeMoss, Silvia Sapora, Jakob Foerster, Nick Hawes, and Ingmar Posner. The complexity dynamics of grokking. *arXiv preprint arXiv:2412.09810*, 2025.

Jesse Hoogland, George Wang, Alexander Gietelink Oldenziel, and Daniel Murfet. Compressibility measures complexity: Minimum description length meets singular learning theory. *arXiv preprint*, 2025.

Simon Pepin Lehalleur, Jesse Hoogland, Matthew Farrugia-Roberts, Susan Wei, Alexander Gietelink Oldenziel, George Wang, Liam Carroll, and Daniel Murfet. You are what you eat—AI alignment requires understanding how data shapes structure and generalisation. *Preprint, under review*, 2025.

Simon Pepin Lehalleur and Richárd Rimányi. Geometry of the fibers of the multiplication map of deep linear neural networks. *arXiv preprint arXiv:2411.19920*, 2025.

Lénaïc Chizat, Edouard Oyallon, and Francis Bach. On lazy training in differentiable programming. *Advances in Neural Information Processing Systems*, 32, 2019.

Bilal Chughtai, Lawrence Chan, and Neel Nanda. A toy model of universality: Reverse engineering how networks learn group operations. *arXiv preprint arXiv:2302.03025*, 2023.

Clémentine C. J. Dominé, Nicolas Anguita, Alexandra M. Proca, Lukas Braun, Daniel Kunin, Pedro A. M. Mediano, and Andrew M. Saxe. From lazy to rich: Exact learning dynamics in deep linear networks. *International Conference on Learning Representations*, 2025.

Jesse Hoogland, George Wang, Matthew Farrugia-Roberts, Liam Carroll, Susan Wei, and Daniel Murfet. Developmental landscape of in-context learning. *arXiv preprint arXiv:2402.02364*, 2024.

Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in Neural Information Processing Systems*, 31, 2018.

Louis Jaburi. Data (attribution) for alignment. *Iliad Intensive lecture notes*, April 2026.

Tanishq Kumar, Blake Bordelon, Samuel J. Gershman, and Cengiz Pehlevan. Grokking as the transition from lazy to rich training dynamics. *International Conference on Learning Representations*, 2024.

Aitor Lewkowycz and Guy Gur-Ari. On the training dynamics of deep networks with L_2 regularization. *Advances in Neural Information Processing Systems*, 33, 2020.

Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. *arXiv preprint arXiv:2301.05217*, 2023.

Xavier Poncini. Computational mechanics for logits. *PIBBSS Report*, October 2025.

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

Andrew M. Saxe, James L. McClelland, and Surya Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *International Conference on Learning Representations*, 2014.

Cosma Rohilla Shalizi and Kristina Lisa Klinkner. Blind construction of optimal nonlinear recursive predictors for discrete sequences. In *Proceedings of the 20th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 504–511, 2004.

Dashiell Stander, Qinan Yu, Honglu Fan, and Stella Biderman. Grokking group multiplication with cosets. *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 46441–46467. PMLR, 2024.

Vikrant Varma, Rohin Shah, Zachary Kenton, János Kramár, and Ramana Kumar. *Explaining Grokking Through Circuit Efficiency.* 2023.

Sumio Watanabe. *Algebraic Geometry and Statistical Learning Theory.* Cambridge University Press, 2009.

William Wu, Lukas Jaburi, Josh Drori, and Jesse Gross. Towards a unified and verified understanding of group-operation networks. *arXiv preprint*, 2024.

Chi-Hang Yip, Rajashree Agrawal, Lawrence Chan, and Jesse Gross. Modular addition without black-boxes: Compressing explanations of MLPs that compute numerical integration. *arXiv preprint*, 2024.